

编程基础 I — 编程语言概述

软件工程与计算 I · 2026 · 南京大学

刘钦

知识点

- 编译和执行
- 编译器的结构
- 不同的编程范式
- 代码是用来读的
- HelloWorld

Outline

- 编程语言
- 计算机硬件、编译器、源代码
- 编译和执行
- 编译器的结构
- 代码与语言
- Lambda-Calculus 和 Java
- Hello World
- Hello Work
- AI4Coding

编程语言

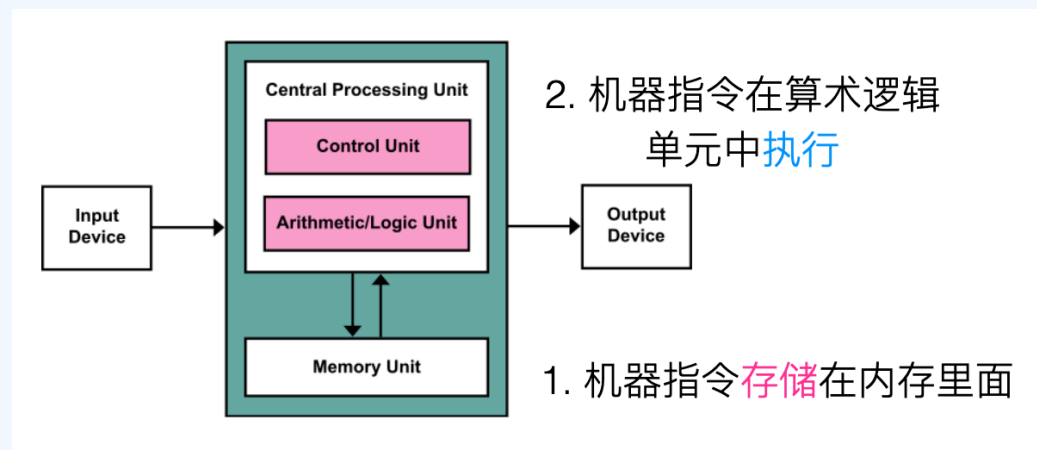
冯诺依曼结构

计算机只能执行**机器指令**

1. 机器指令**存储**在内存里
2. 机器指令在**算术逻辑单元**中执行

输入 → 存储 → 执行 → 输出

所有编程语言，最终都要转化为
机器能理解的指令序列



编译和执行

方式	语言代表	流程
直接解释执行	Basic	源代码 → 解释器 → 目标机器
编译、执行	C	源代码 → 编译器 → 机器指令 → 目标机器
编译成字节码、解释执行	Java	源代码 → 编译器 → 字节码 → 虚拟机 → 目标机器

编译与解释

编译 Compile: 把整个程序源代码翻译成另外一种代码，然后等待被执行，发生在运行之前，产物是「另一份代码」。

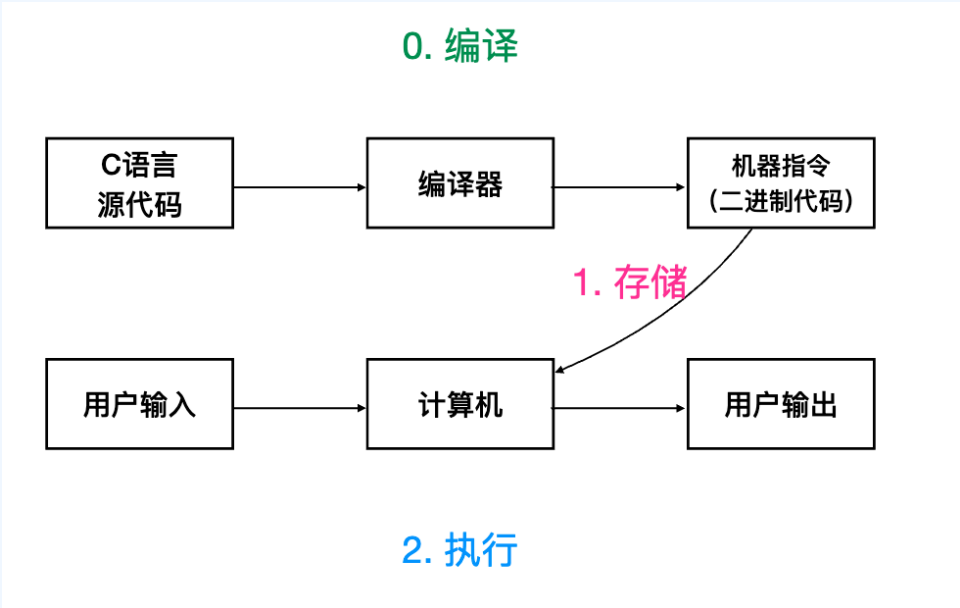
解释 Interpret: 把程序源代码一行一行地读懂然后执行，发生在运行时，产物是「运行结果」。

<http://huang-jerryc.com/2016/11/20/do-you-know-the-different-between-compiler-and-interpreter/>

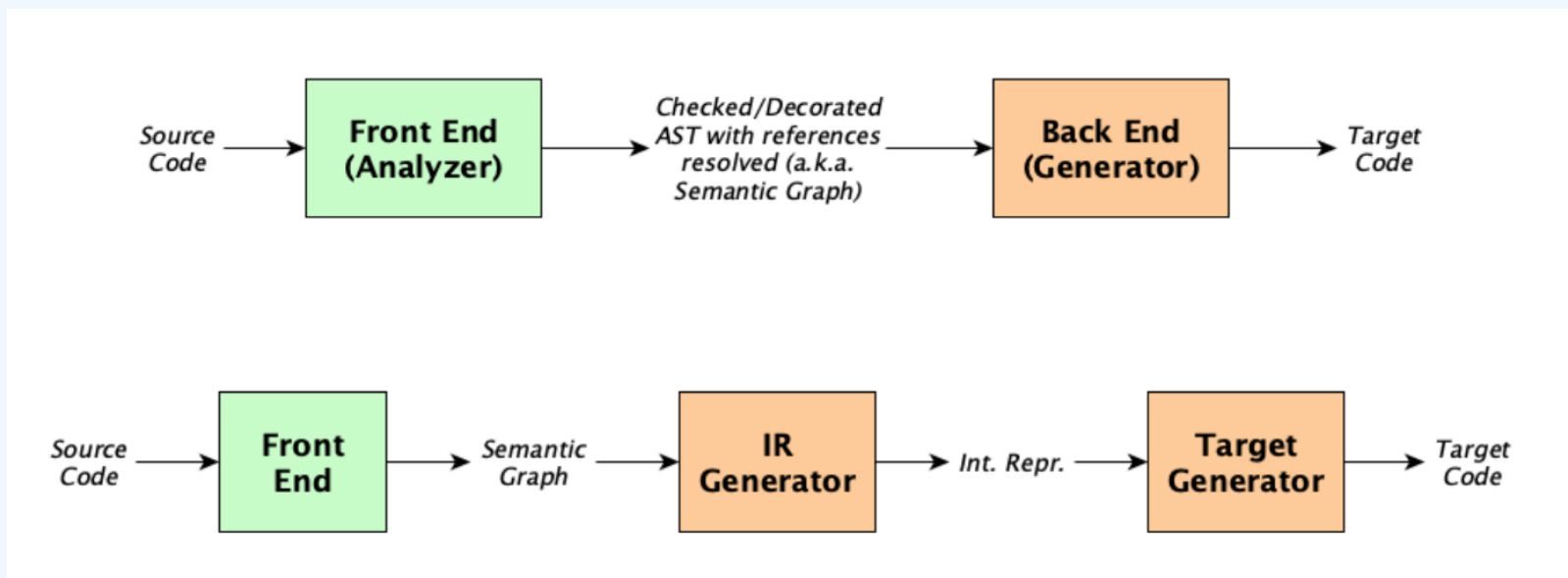
编译器的结构

输入：源程序 · 输出：目标代码

阶段	说明
词法分析	字符流 → Token
语法分析	Token → AST
语义分析	类型检查、符号表
中间代码生成	机器无关 IR
代码优化	删除冗余、强度削弱
目标代码生成	IR → 机器指令



编译器前端 / 后端架构



词法分析 (Lexical Analysis)

扫描输入符号，将其分组形成具有一定含义的单元

输入: `temp := x + 1`

输出 (Token 序列):

temp	:=	x	+	1
标识符	赋值	标识符	运算符	整数

【AI视角】LLM 的 Tokenizer vs 词法分析

编译器：词法分析

```
temp := x + 1
```

Token	类型
temp	标识符
:=	赋值符
x	标识符
+	运算符
1	整数字面量

规则由**语法规则**定义

LLM：Tokenizer (BPE)

```
temperature := x + 1
```

Token	ID
temp	7123
erature	2840
:=	3345
x	220
+	648
1	352

规则由**训练数据频率**决定

关键差异：编译器的词法分析基于语义规则，LLM 的 Tokenizer 基于统计——temperature 被切成 temp + erature，不理解含义。

语法分析 (Syntactic Analysis)

单词符号被分组成语义单元，输出解析树

```
result := salary + bonus * 1.10
```

```
      :=  
     /  \  
  result  +  
         / \  
      salary *  
          / \  
        bonus 1.10
```

语义分析 (Semantic Analysis)

- 语义检查: `Temp` 和 `x` 的类型由声明决定, 不同声明语义不同
- `*` 乘法: 可以是整数乘法, 也可以是浮点数乘法

输出: 扩展的解析树, 包含:

- 标识符的类型
- 标识符在哪里声明

语法树的执行过程

代码优化

常见优化手段

- 复写传播
- 代码外提
- 删除无用代码
- 强度削弱
- 常量合并
- 变换循环控制条件
- 代码移动
- 合并已知变量
- 内联函数的调用
- 删除公共子表达式

中间代码 (Intermediate Representation)

序号	指令类型	指令形式
1	赋值指令	$x = y \text{ op } z$ $x = \text{op } y$
2	复制指令	$x = y$
3	条件跳转	if $x \text{ relop } y$ goto n
4	非条件跳转	goto n
5	参数传递	param x
6	过程调用	call p, n
7	过程返回	return x
8	数组引用	$x = y[i]$
9	数组赋值	$x[i] = y$
10	地址及指针操作	$x = \& y$ $x = *y$ $*x = y$

➤ $x = y \text{ op } z$ (**op** , y , z , x)
➤ $x = \text{op } y$ (**op** , y , $_$, x)
➤ $x = y$ (**=** , y , $_$, x)
➤ if $x \text{ relop } y$ goto n (**relop** , x , y , n)
➤ goto n (**goto** , $_$, $_$, n)
➤ param x (**param** , $_$, $_$, x)
➤ call p, n (**call** , p , n , $_$)
➤ return x (**return** , $_$, $_$, x)
➤ $x = y[i]$ (**=[]** , y , i , x)
➤ $x[i] = y$ (**[]=** , y , x , i)
➤ $x = \&y$ (**&** , y , $_$, x)
➤ $x = *y$ (**=*** , y , $_$, x)
➤ $*x = y$ (***=** , y , $_$, x)

目标代码生成

- **Target language**: 了解目标语言的性质, CISC 或 RISC 架构
- **IR Type**: 中间表示可以是 AST、反向波兰表示法或 3 地址代码
- **Instruction selection**: 将中间表示映射到目标机器指令集
- **Register allocation**: 决定哪些值保留在寄存器中
- **Instruction ordering**: 为指令创建时间表以执行

目标代码 — X86

```
int foo(int a, int b){ return a + b + 10; }
```

序曲

```
pushq %rbp
```

```
movq %rsp, %rbp
```

函数体

```
movl %edi, -4(%rbp)    # 把第1个参数写到栈里第一个位置
```

```
movl %esi, -8(%rbp)    # 把第2个参数写到栈里第二个位置
```

```
movl -4(%rbp), %eax    # 把第1个参数写到 %eax 寄存器
```

```
addl -8(%rbp), %eax    # 把第2个参数加到 %eax
```

```
addl $10, %eax         # 把立即数10加到 %eax (同时也是返回值)
```

尾声

```
popq %rbp
```

```
retq
```

目标代码 — ARM

```
int foo(int a, int b){ return a + b + 10; }
```

```
// 序曲：把栈扩展8个字节，用于放两个参数
```

```
sub sp, sp, #8
```

```
// 函数体
```

```
str r0, [sp, #4]    // 把第1个参数写到栈顶+4的位置
```

```
str r1, [sp]        // 把第2个参数写到栈顶位置
```

```
ldr r0, [sp, #4]    // 把第1个参数加载到 r0
```

```
ldr r1, [sp]        // 把第2个参数加载到 r1
```

```
add r0, r0, r1      // r1 加到 r0, 结果保存在 r0
```

```
add r0, r0, #10     // 常量10加到 r0 (r0 也是返回值)
```

```
// 尾声
```

```
add sp, sp, #8
```

```
bx  lr
```

代码与语言

同一件事，可以用不同语言表达：

- 用汉语写出《山居秋暝》
- 用Python 语言写出 `print("hello, world")`

语言是思想的载体。
编程语言决定了你如何思考程序。

编程语言历史

编程语言流行度 (TIOBE · Feb 2026)

Feb 2026	Feb 2025	Change	Programming Language		Ratings	Change
1	1			Python	21.81%	-2.08%
2	4	▲		C	11.05%	+1.22%
3	2	▼		C++	8.55%	-2.82%
4	3	▼		Java	8.12%	-2.54%
5	5			C#	6.83%	+2.71%
6	6			JavaScript	2.92%	-0.85%
7	10	▲		Visual Basic	2.85%	+0.81%
8	15	▲		R	2.19%	+1.14%
9	7	▼		SQL	1.93%	-0.93%
10	9	▼		Delphi/Object Pascal	1.88%	-0.29%
11	30	▲		Perl	1.67%	+1.19%
12	11	▼		Fortran	1.64%	-0.12%
13	14	▲		PHP	1.34%	+0.20%
14	13	▼		Rust	1.32%	-0.14%
15	12	▼		Scratch	1.30%	-0.25%

编程语言历史 (1950s–1970s)

- 1951 — Regional Assembly Language
- 1952 — Autocode
- 1954 — IPL (LISP 前身)
- 1955 — FLOW-MATIC (COBOL 前身)
- **1957 — FORTRAN** (第一个编译器)
- 1958 — LISP
- 1958 — ALGOL 58
- 1959 — COBOL
- 1959 — RPG
- 1962 — APL
- **1962 — Simula**
- 1963 — CPL (C 前身)
- **1964 — BASIC**
- 1964 — PL/I
- 1967 — BCPL (C 前身)

Establishing Paradigms (1970s)

- 1969 — B (C 前身)
- **1972 — C** (结构化编程)
- **1972 — Smalltalk** (面向对象)
- **1972 — Prolog** (逻辑编程)
- **1975 — Scheme** (函数式)
- **1978 — SQL** (DSL)

编程语言历史 (1980s–2000s)

1980s 巩固期

- 1980 — C++
- 1983 — Ada
- 1986 — Objective-C
- 1987 — Perl
- 1989 — FL (Backus)

1990s 互联网时代

- 1990 — Haskell
- 1991 — Python
- 1995 — Java
- 1995 — JavaScript
- 1995 — PHP

2000s 多范式

- 2001 — C#
- 2003 — Scala
- 2005 — F#
- 2007 — Clojure
- 2009 — Go

2010s 移动时代

- 2011 — Kotlin
- 2012 — TypeScript
- 2014 — Swift
- 2015 — Rust

Current Trend

- 2021 — Power Fx
- 2022 — Carbon
- 2023 — Mojo
- 华为: 仓颉 (Cangjie) 和 ArkTS

编程范式

编程范式总览

Imperative (命令式)

- Procedure (过程式)
- Object-Oriented (面向对象)

Declarative (声明式)

- Functional (函数式)
- Logic (逻辑式)
- DSL (领域特定语言)

【AI视角】AI 更"喜欢"哪种范式?

AI 容易理解

函数式

- 纯函数，无副作用，输入决定输出
- `factorial(5)` 永远返回 120

声明式 / DSL

- 意图明确，冗余少
- SQL、正则表达式对 LLM 友好

小而内聚的类 (OOP)

- 单一职责，命名清晰

AI 容易出错

命令式 (复杂状态)

- 多个可变变量、复杂循环
- AI 容易遗漏状态转换

隐式上下文依赖

- 全局变量、隐式 this
- AI 无法"感知"运行时状态

并发 / 多线程

- 竞争条件、锁的使用
- AI 生成的并发代码往往有缺陷

结论：代码写得越"函数式"、越"声明式"，AI 辅助效果越好。

Imperative vs Declarative

命令式 (HOW): 我看到 Gone Fishin' 标牌下面那张桌子是空的, 我和我丈夫要走过去坐下。

声明式 (WHAT): 两人桌, 谢谢。

命令式 (Imperative) — C# 示例

```
Dictionary<string, Grouping> groups = new Dictionary<string, Grouping>();
foreach (Product p in products)
{
    if (p.UnitPrice >= 20)
    {
        if (!groups.ContainsKey(p.CategoryName))
        {
            Grouping r = new Grouping();
            r.CategoryName = p.CategoryName;
            r.ProductCount = 0;
            groups[p.CategoryName] = r;
        }
        groups[p.CategoryName].ProductCount++;
    }
}
List<Grouping> result = new List<Grouping>(groups.Values);
result.Sort(delegate(Grouping x, Grouping y) {
    return x.ProductCount > y.ProductCount ? -1 :
           x.ProductCount < y.ProductCount ? 1 : 0;
});
```

声明式 & DSL (Declarative) — LINQ 示例

// LINQ 方法语法

```
var result = products
    .Where(p => p.UnitPrice >= 20)
    .GroupBy(p => p.CategoryName)
    .OrderByDescending(g => g.Count())
    .Select(g => new { CategoryName = g.Key, ProductCount = g.Count() });
```

// DSL: LINQ 查询语法

```
var result =
    from p in products
    where p.UnitPrice >= 20
    group p by p.CategoryName into g
    orderby g.Count() descending
    select new { CategoryName = g.Key, ProductCount = g.Count() };
```

函数式 (Functional) — 阶乘对比

C — 命令式

```
int factorial(int n) {  
    int f = 1;  
    for (; n > 0; --n)  
        f *= n;  
    return f;  
}
```

描述**怎么算** (循环、变量、赋值)

Lisp — 函数式

```
(defun factorial(n)  
  (if (= n 0)  
      1  
      (* n (factorial(- n 1)))))  
; 若 n=0, 则 n!=1  
; 否则 n! = n * (n-1)!
```

描述**是什么** (递归定义)

面向对象 (Object-Oriented)

目标：可修改性

通讯机制：消息机制

`(15 * 19) + (37 squared)`

向 15 发送消息 `*`，参数为 19；

向 37 发送消息 `squared`；

最后向 `15*19` 的结果发送消息 `+`，参数为 `37 squared` 的结果。

过程式 vs 面向对象

过程式

```
起床开始
    openEye;      // 睁眼, 开始到结束
    dressed;      // 穿衣服, 开始到结束
    getOutOfBed;  // 下床, 开始到结束
起床结束
```

面向对象

```
class People {
    public void getUp() {
        eye.open();      // 睁眼
        body.dressed();   // 穿衣服
        body.getOutOfBed();// 下床
    }
}
```

逻辑范式 (Prolog)

```
man(socrates).  
mortal(X) :- man(X).  
  
?- mortal(socrates).    % → Yes
```

- 事实: `man(socrates)` — 苏格拉底是人
- 规则: `mortal(X) :- man(X)` — 所有人都是凡人
- 查询: `mortal(socrates)` — 苏格拉底是否是凡人?

阶乘 (Prolog 逻辑式)

```
factorial(0, 1).  
factorial(N, F) :-  
    M is N-1, factorial(M, Fm), F is N * Fm.
```

Lambda-Calculus 和 Java

Alonzo Church 与 Lambda-Calculus

- **1930s**: Alonzo Church 提出了 Lambda-Calculus 理论
- Shengyi Wang 开发了一个 Lambda 解释器 (Haskell 编写)
 - <https://github.com/txyyss/Lambda-Calculus/releases/tag/v1.0>
 - 支持 Mac、Windows 系统

安装 Lambda 解释器

```
sudo chmod 777 lambda    # Mac
./lambda                  # 启动 REPL

lambda.exe                # Windows
```

What is Java?

Java 不仅仅是编程语言，还包括**软件平台**：

- Java Card
- Java ME
- **Java SE**
- Java EE

跨平台技术

Java 源代码
↓ javac
字节码 (bytecode)
↓ JVM (各平台)
目标机器执行

- JVM 自身的实现不是跨平台的
- JVM 解释执行 + JIT 编译

Java 的诞生

May 23, 1995 — First public release

At that time, the entire Java technology team, not yet a division, numbered less than 30 people. It was the original members of this small group who created and nurtured a technology that would change the computing world.

创造者: **James Gosling**

前身: OAK (1991 Green Team: *7 device, 失败)

Java meets the Internet

In June and July 1994, after three days of brainstorming with John Gage, the Director of Science for Sun, Gosling, Joy, Naughton, Wayne Rosing, and Eric Schmidt, the team **re-targeted the platform for the World Wide Web**.

- Naughton 写了小型浏览器 **WebRunner** (后改名 HotJava)
- 感受到 Mosaic 浏览器带来的互联网时代到来

改变世界的两个 Demo

Demo 1 — 旋转分子

Gosling 将鼠标移到插图中间的 3D 分子上，分子随鼠标旋转。

"The entire audience went 'Aaaaaaah!'" — 他们对现实的认知彻底改变了。

Demo 2 — 动态排序算法

三组随机长度的水平线，在浏览器中被可视化地从短到长排序，线条真的在移动！

突然，房间里的每个人都开始重新思考互联网的潜力。

世纪之战：Java vs Android

时间线

时间	事件
2010.8	Oracle 起诉 Google，称 Android 侵犯 Java 版权
2012.5	陪审团认定部分侵权，但法官 William Alsup 推翻
2014.4	联邦上诉法院判定 API 受版权保护
2015.6	最高法院拒绝介入，维持原判
2016.5	地区法院再次开庭，审理是否属于"合理使用"
2021.4	最高法院以 6:2 判决 Google 对 Java API 的使用属于合理使用

学习编程的法官

"Alsup was the presiding judge over Oracle America, Inc. v. Google, Inc., where he notably **learned the Java programming language solely for the purpose of being able to understand the case** more clearly."

— https://en.wikipedia.org/wiki/William_Haskell_Alsup

API 的价值争论：

如果 API 受版权保护，下游企业将不得不故意采用不兼容的标准以免面临诉讼风险。这与当前促进不同服务间互操作性的行业趋势是不兼容的。

Java 版本历史

版本	发布时间
Java SE 7	2011.7.28
Java SE 8	2014.3.8 (LTS)
Java SE 9	2016.9.22
Java SE 11	2018.9 (LTS)
Java SE 17	2021.9 (LTS)

版本	发布时间
Java SE 19	2022.9
Java SE 20	2023.3
Java SE 21	2023.9 (LTS)
Java SE 22	2024.3
Java SE 23	2024.9
Java SE 24	2025.3
Java SE 25	2025.9
Java SE 26	2026.3

Hello World

HelloWorld — 第一个 Java 程序

```
// Hello World in Java
class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

编译 & 运行

```
javac HelloWorld.java    # 编译 → 生成 HelloWorld.class
java HelloWorld          # 运行
```

解析 HelloWorld

.java 文件与 class 定义

- `class` 关键字
- 只能有一个 `public class`
- `public class` 名字必须与文件名相同

main 方法

```
public static void main(String[] args) {}
```

每个应用都要有这个入口方法

在终端打印

```
System.out.println("Hello World!");
```

Java 注释

```
// 单行注释（到行尾）
```

```
/* 多行注释 */
```

```
/** 文档注释（javadoc 使用） */
```

【AI视角】 AI 辅助写 Hello World

实验：用通义灵码 / 文心快码完成以下任务

任务 1：生成

在 IDE 中输入注释，让 AI 补全：

```
// 写一个打印 Hello World 的 Java 程序
```

观察：AI 生成的代码和你预期的一样吗？

任务 2：解释

选中代码，问 AI：

"解释这段代码每一行的含义"

关键体验：AI 是"知道答案"还是"理解了问题"？

任务 3：出错了怎么办

故意写错：

```
System.out.Println("Hello"); // 大写P
```

让 AI 帮你找 bug。

任务 4：跨语言

让 AI 把 Java 版本翻译成 Python、C、JavaScript。

观察 4 种语言的**结构差异**。

代码是用来读的

Write once, Read many times

"When you program, you have to think about how someone will read your code, not just how a computer will interpret it."

— Kent Beck

代码可读的原因：

- 团队的需要
- 维护的需要

糟糕的变量名字

```
x = x - xx;  
xxx = aretha + SalesTax(aretha);  
x = x + LateFee(x1, x) + xxx;  
x = x + Interest(x1, x);
```

良好的变量名字

```
balance = balance - lastPayment;  
monthlyTotal = newPurchases + SalesTax(newPurchases);  
balance = balance + LateFee(customerID, balance) + monthlyTotal;  
balance = balance + Interest(customerID, balance);
```

变量名应该表达意图，而不是隐藏意图。

【AI视角】代码可读性 × AI

对 AI 训练的影响

GitHub 上的代码是 LLM 的训练数据：

- 好代码 → AI 学会正确模式
- 烂代码 → AI 学会坏习惯

`x = x - xx` 这样的代码如果大量存在于训练集，AI 也会输出同样难读的命名。

AI 生成代码的可读性问题

AI 常见的"反模式"：

```
// AI 生成（可运行，但难读）
int r = 0;
for(int i=0;i<n;i++) r+=a[i];
return r/n;

// 应该是
int sum = 0;
for (int i = 0; i < count; i++) {
    sum += scores[i];
}
return sum / count;
```

AI 时代的新要求：不只要能写代码，更要能审查 AI 生成的代码。

请使用英文法语符号

输入时注意区分：

- 中文符号：, 。 ; () " " — 不能用于代码
- 英文符号：, . ; () " " — 代码中必须使用

不同语言的 Hello World

如果你想看看 603 种编程语言 + 78 种人类语言的 Hello World:

| <http://helloworldcollection.de/>

换行与缩进

```
// 一般：一条语句换一行  
balance = balance - lastPayment;
```

```
// 有内在逻辑关系要换行  
// 缩进表示层级关系  
f() {  
    if (condition) {  
        doSomething();  
    }  
}
```

Hello Work

Top Java Projects on GitHub

- **RxJava** — Reactive Extensions for JVM
- **Elasticsearch** — 搜索引擎
- **Spring Boot** — 微服务框架
- **Guava** — Google Core Libraries
- **Netty** — 网络框架
- **Jenkins** — CI/CD
- **MyBatis** — ORM 框架
- **Retrofit** — HTTP 客户端
- **OkHttp** — HTTP 客户端
- **Hystrix** — 熔断器
- **Fastjson** — JSON 解析
- **libGDX** — 游戏开发
- **ZXing** — 二维码
- **Mockito** — 单元测试

Java 开源项目 — 阿里巴巴

- **Arthas** — Java 诊断工具
- **Druid** — JDBC 组件库（数据库连接池）
- **fastjson** — JSON 解析库
- **EasyExcel** — 省内存的 Excel 读写
- **canal** — MySQL binlog 增量订阅
- **Nacos** — 服务发现、配置管理
- **Sentinel** — 流量控制、熔断降级
- **Spring Cloud Alibaba** — 微服务一站式解决方案
- **P3C** — Java 规约检查插件（IDEA/Eclipse）
- **COLA** — Clean Object-Oriented and Layered Architecture

AI for Coding

初次震撼 — AlphaCode（2022年2月）

<https://alphacode.deepmind.com/>

它做到了什么？

- 参加 **Codeforces** 竞技编程比赛
- 在模拟赛中达到 **前 50%** 水平，超越大多数人类参赛者
- 解题类型涵盖：搜索、动态规划、图论、数学推导

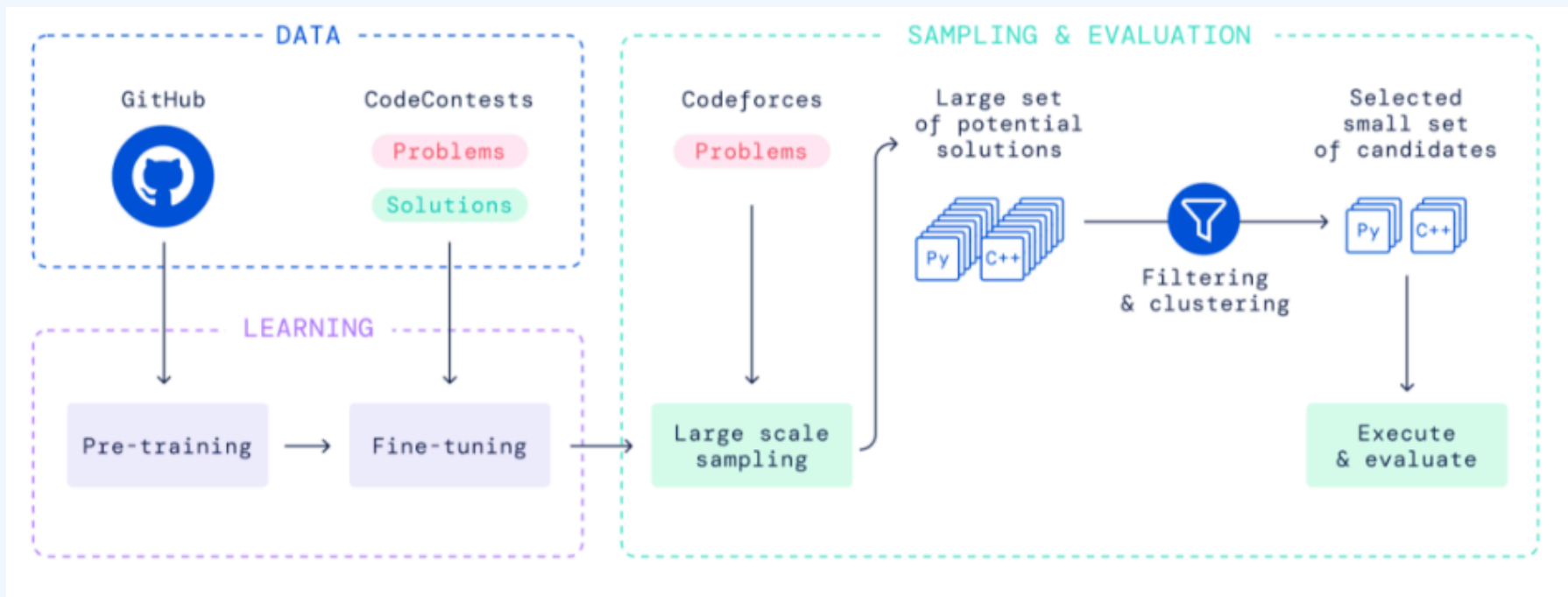
为何震撼？

编程竞赛题需要理解自然语言题意 → 建模 → 设计算法
→ 正确实现
这是此前认为 AI 最难突破的综合推理链

关键数据

指标	数值
模型参数量	41B（针对代码微调）
训练代码量	715GB GitHub 代码
每题采样数	1000 个候选解
过滤策略	运行测试 + 聚类去重，取最优
竞赛排名	模拟赛前 49.9%

AlphaCode 架构



① **题意理解** — Encoder 编码自然语言题目 ② **大规模采样** — Decoder 生成 1000 个候选程序 ③ **过滤 + 聚类** — 运行示例测试，去重选优

初步迈向实用性 — Copilot（2022—2023）

<https://github.com/features/copilot>

从竞赛到日常开发

时间	里程碑
2021.06	Technical Preview（受邀测试）
2022.06	正式发布，每月 \$10
2023.03	Copilot X — 接入 GPT-4
2023.05	Copilot Chat 公测，对话式编程
2024	企业版 100 万+ 付费用户

实测影响

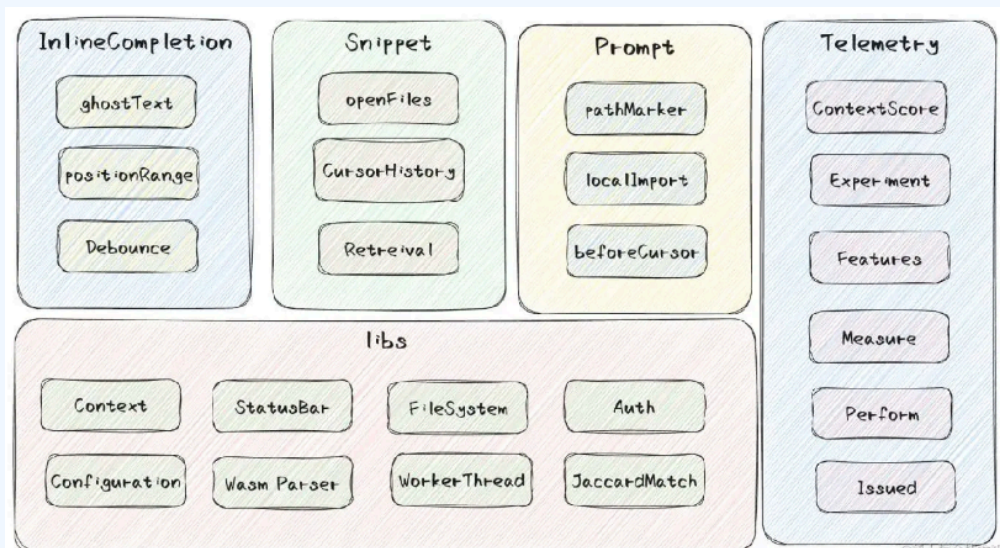
- 用户接受率约 30% 的补全建议
- 已使用 Copilot 的开发者中 46% 的代码 由 AI 生成
- 任务完成速度平均快 55%（GitHub 调研）
- 支持 VS Code / JetBrains / Vim / Neovim

底层技术

OpenAI Codex — GPT-3 在 GitHub 公开代码上微调
输入：光标前后上下文 → 输出：下一段代码

Copilot 架构模块深解

参考: <https://juejin.cn/post/7301573790342594575>



① InlineCompletion — 触发与展示

ghostText 幽灵文字渲染 · positionRange 插入位置 · Debounce 防抖节流

② Snippet — 上下文检索

openFiles 已打开文件片段 · CursorHistory 光标轨迹 · Retrieval 向量召回

③ Prompt — 构造最终输入

pathMarker 文件路径提示 · localImport 提取 import · beforeCursor/afterCursor 光标前后文

④ Telemetry — 效果闭环

ContextScore 上下文质量评估 · Experiment A/B 分组 · Features 功能灰度

⑤ libs — 基础服务层

Auth 鉴权 · FileSystem 文件访问 · StatusBar 状态栏 · Context 共享上下文

项目级实用性

<https://claude.ai/code>

不只是补全，而是协作开发

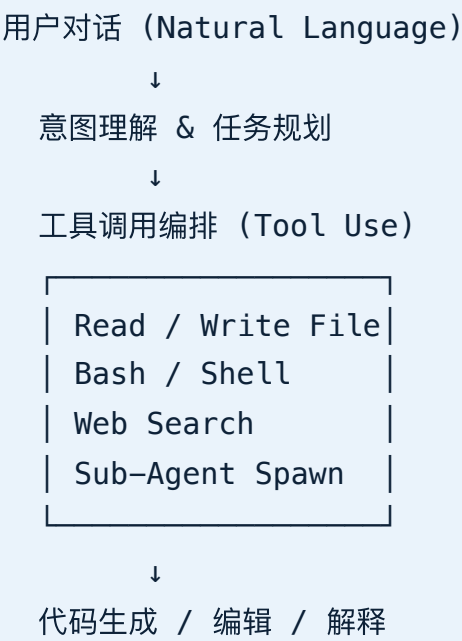
- **自然语言驱动** — 用对话描述需求，直接生成、修改代码
- **全项目上下文** — 理解整个代码库，跨文件重构
- **执行工具调用** — 读写文件、运行命令、调用 API
- **多步骤任务** — 分解复杂需求，自主完成端到端实现

典型使用场景

场景	示例
新功能开发	"帮我加一个用户登录模块"
Bug 修复	"这个函数为什么报错？修一下"
代码审查	"Review 这个 PR，找出潜在问题"
文档生成	"给这个模块写 README"
测试编写	"为所有公开方法生成单元测试"

Claude Code 架构

核心能力层



与 Copilot 的区别

	Copilot	Claude Code
粒度	行 / 函数	项目级
交互	补全触发	对话驱动
上下文	当前文件	整个代码库
执行	只生成	可运行命令
自主性	低	高 (Agent)

中国编程大模型全景

开源阵营

模型	机构	亮点
DeepSeek-Coder-V2	DeepSeek	代码/数学双强，开源最强之一
Qwen2.5-Coder	阿里通义	32B 版本达 GPT-4o 水平
CodeGeeX4	清华 & 智谱	专为 IDE 补全优化，VS Code 插件
MiniCPM-Code	面壁智能	端侧部署，手机可跑

闭源 / 商业

模型	机构	亮点
通义灵码	阿里云	企业级，深度集成钉钉/云效
文心快码	百度	接入文心 4.0，支持低代码平台
Kimi	月之暗面	长上下文 128K，擅长读代码库

与国际模型对比

维度	国产代表	国际代表
代码补全	CodeGeeX4	GitHub Copilot
代码生成	DeepSeek-Coder-V2	GPT-4o / Claude 3.5
项目级 Agent	通义灵码	Claude Code / Cursor
端侧部署	MiniCPM-Code	——
开源生态	Qwen2.5-Coder	CodeLlama / StarCoder2

为何关注国产模型？

- 数据合规：代码不出境，企业采购首选
- 中文注释理解更准确，文档生成质量更高
- 本土生态：与国内 DevOps 平台深度集成
- SEECODER 正是在此背景下探索的研究方向

【AI视角】 Prompt 是一种编程语言吗？

编程语言的特征

特征	传统语言	Prompt
语法规则	严格	模糊
确定性	相同输入→相同输出	有随机性
执行环境	编译器/VM	LLM
调试手段	编译器报错	反复试探
抽象能力	函数、类	Few-shot示例

范式对应

命令式 Prompt

第一步：提取所有名词。

第二步：检查是否有动词。

第三步：输出结果。

声明式 Prompt

用一句话概括这段文字的核心观点。

函数式 Prompt (Few-shot)

输入: "cat" → 输出: "动物"

输入: "table" → 输出: "家具"

输入: "Python" → 输出: ?

结论：Prompt 更接近**声明式语言**——你描述**想要什么**，而不是**怎么做**。但它缺乏确定性，这是与传统编程语言的本质区别。

【AI视角】 AI 编程的局限性

① 业务逻辑的模糊性陷阱

"实现一个计算员工奖金的方法，要求公平合理"

AI 会基于**隐含假设**填补空白：

- 请假怎么算？试用期呢？绩效系数呢？
- 生成的代码"能跑"，但假设往往与真实需求不符

👉 要学会识别 AI 的隐含假设，而非盲目接受

② 隐蔽的逻辑 Bug

"判断两个浮点数是否相等" / "线程安全的单例模式"

代码表面正确，在特殊场景下失效：

- 浮点精度： `0.1 + 0.2 == 0.3` → `false`
- 并发竞态：double-checked locking 的经典陷阱

👉 设计测试用例，主动攻破 AI 的代码

③ 需要上下文的架构决策

"系统每天处理 100 万订单，设计订单状态流转的数据结构"

AI 不感知：

- 系统现状与历史技术债
- 团队规模与能力边界
- 运维成本与迁移代价

给出的方案往往"正确但不可用"

👉 架构决策需要人类的判断，不能外包给 AI

④ 版本与依赖幻觉 (Hallucination)

让 AI 生成 Spring Security 6.x 的认证配置代码

- AI 可能混淆 5.x / 6.x 的 API 差异
- "自信地"使用**不存在的方法**
- 运行报错，但 AI 解释时仍坚持"代码正确"

👉 AI 的自信度 $\neq$ 正确率，永远需要验证

思考题

什么是更适合 AI 之间交流的语言？

- 人与机器之间交流的语言未来会是怎样的？
- 自然语言 vs 编程语言 vs 某种新形式？
- LLM 时代，编程语言的边界在哪里？